

Université Badji Mokhtar Annaba
Faculté de l'ingéniorat
Département d'informatique
Master 1 Réseaux et sécurité

Pattern Proxy

Un patron structurel

MAA. S.yahiouche

definition

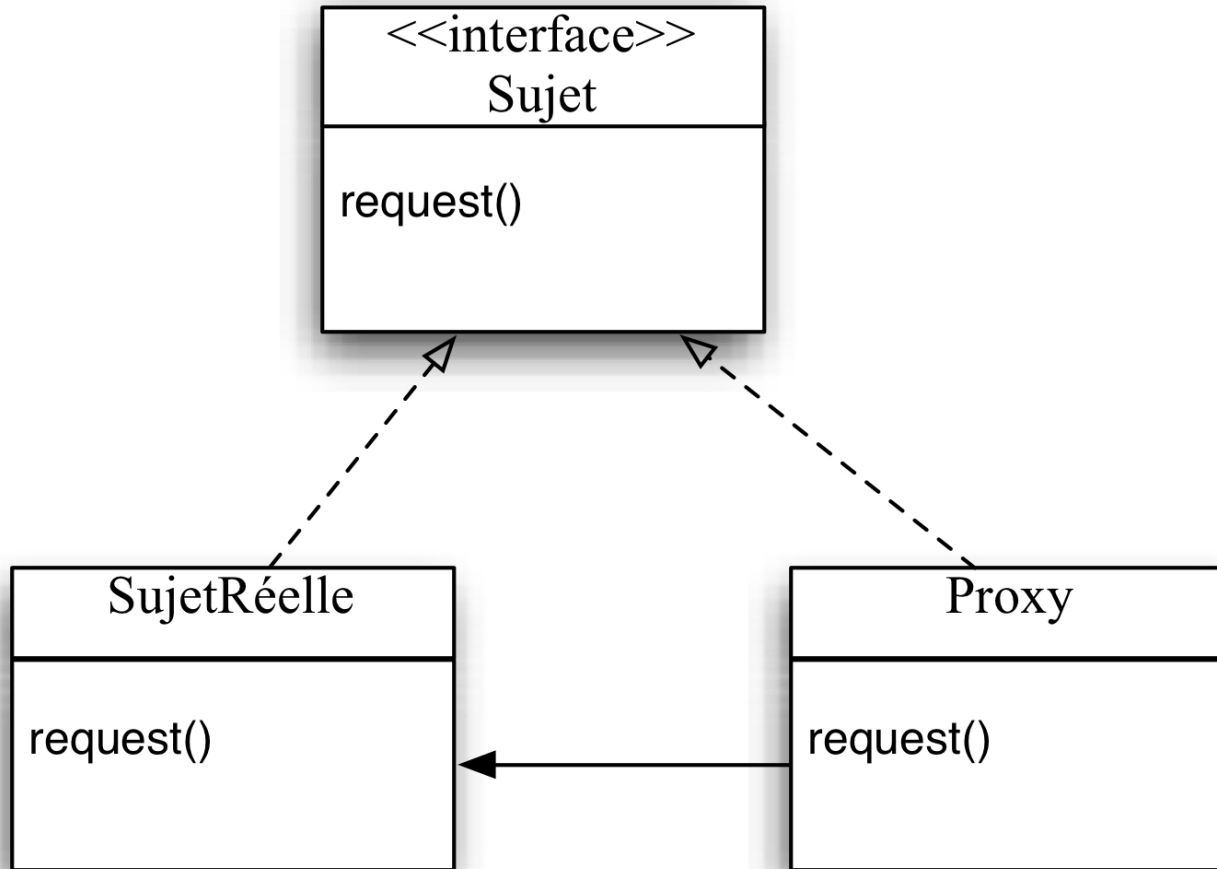
- Un proxy est une classe se substituant à une autre classe.
- Par convention, le proxy implémente la même interface que la classe à laquelle il se substitue.

Différents types de proxy

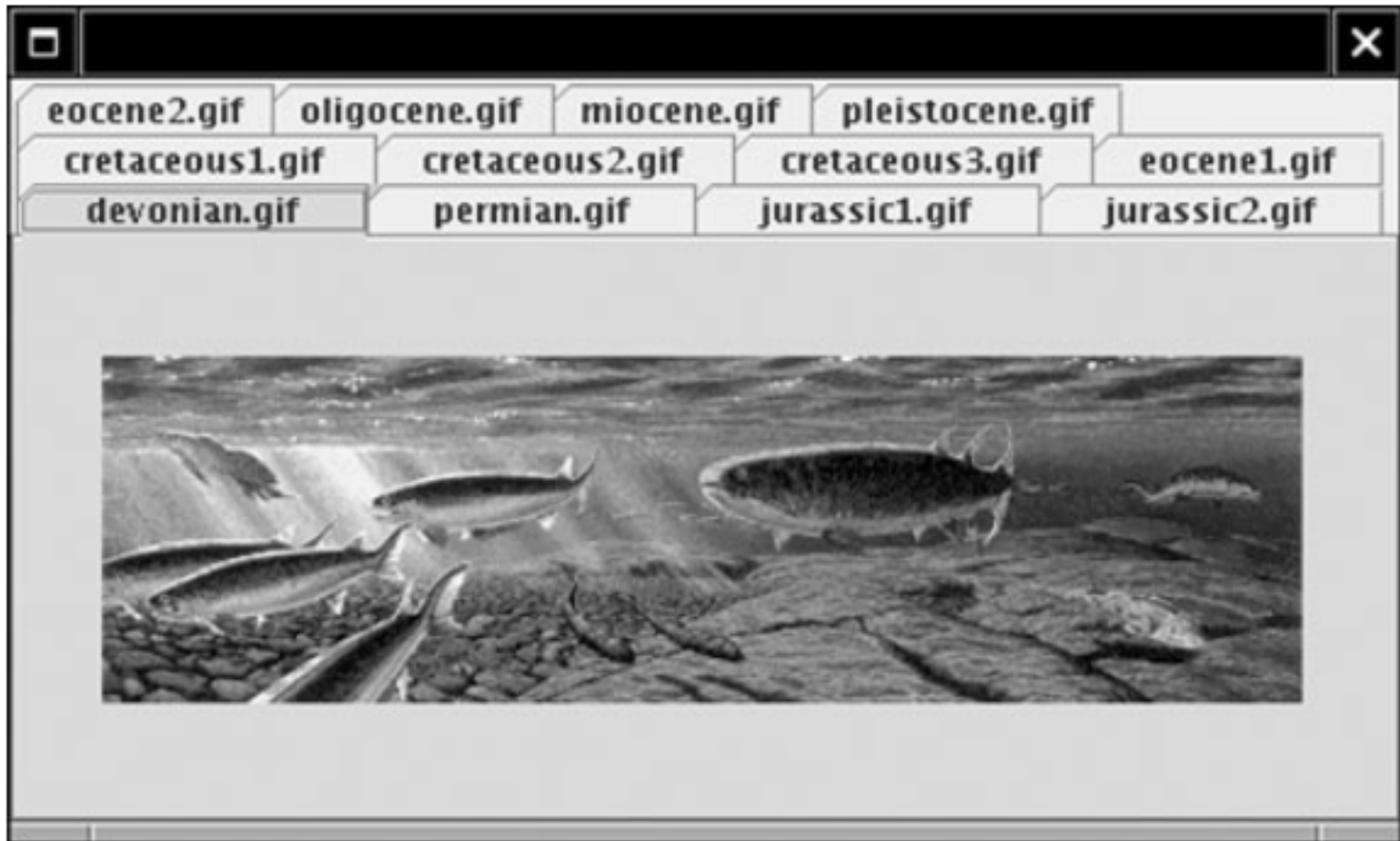
Il existe différents types de Proxy ayant un comportement ou un rôle différent :

- Remote proxy : fournir une référence sur un objet situé sur un espace d'adressage différent, sur la même machine ou sur une autre
- Virtual proxy : retarder l'allocation mémoire des ressources de l'objet jusqu'à son utilisation réelle
- Protection (access) proxy : fournir à chaque classe cliente un accès à l'objet avec des niveaux de protection différents,
- Firewall proxy : protéger l'accès à l'objet par des classes « malveillantes » ou vice-versa,
- Cache proxy : stocker le résultat d'opérations coûteuse en temps, afin de pouvoir les partager avec les différentes classes utilisatrices.

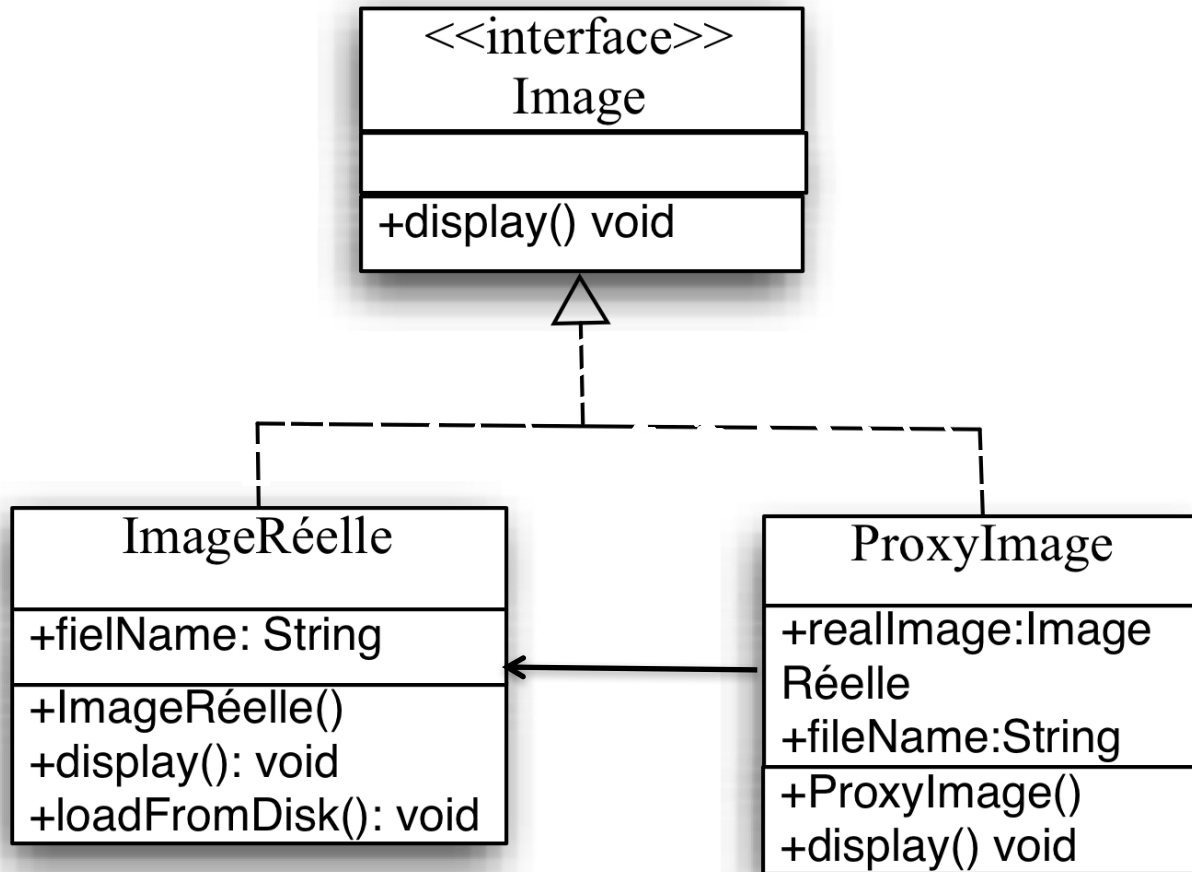
Diagramme de classe



Exemple de Virtual Proxy: lorsque l'image est couteuse à charger



Exemple de Virtual Proxy



Virtual Proxy implémentation: Etape 1

- Créer une interface

```
public interface Image{  
void display();  
}
```

Etap2: implémenter la classe réelle

```
public class ImageRéelle implements Image{  
    private String fileName;  
    public ImageRéelle (String fileName)  
    {This.fileName=fileName;  
    loadFromDisk();  
    }  
    public void display(){  
        System.out.println("Displaying " + fileName);  
    }  
    private void loadFromDisk(){  
        System.out.println("Loading... " + fileName);  
    }  
}
```


Etape 3 implémenter la classe proxy

```
public class ProxyImage implements Image{  
  private ImageRéelle reallImage;  
  private String fileName;  
  public ProxyImage(String fileName){ this.fileName  
    = fileName; }  
  public void display() {  
    if(reallImage == null){  
      reallImage = new ImageRéelle(fileName); }  
    else reallImage.display(); }
```

Etape 4: exécution: l'appel du proxy

```
public class ProxyPatternDemo {  
    public static void main(String[] args) {  
        Image image = new ProxyImage("test_10mb.jpg");  
        //l'image va-t-être chargé depuis le disque:..... Loading  
        image.display();  
  
        //l'image va – t –être affichée: Displaying  
        image.display();  
    }  
}
```

La sortie:

Loading... test_10mb.jpg

Displaying test_10mb.jpg

Protection Proxy: l'interface

- **public interface** Internet {

public void connectTo (**String** serverHost)
throws Exception;}

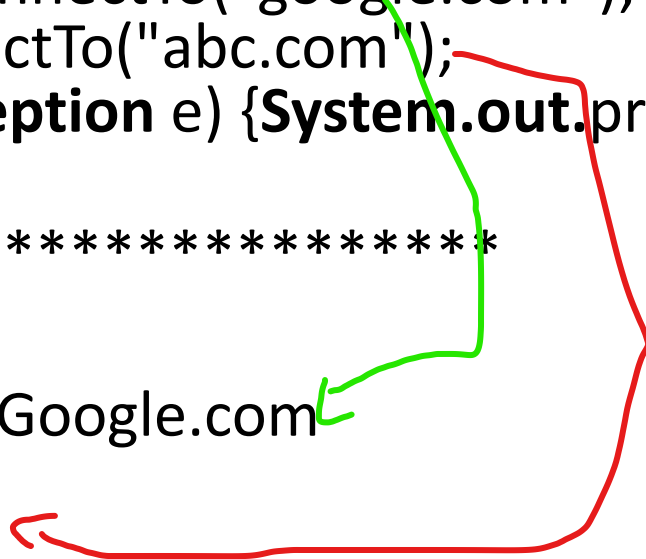
Protection Proxy: Le Client

- **public class** ProtecProxyTester {
public static void main(**String**[] args) **throws** Exception{
Internet inter= new **ProxyInter**();
try { inter.connectTo("google.com");
inter.connectTo("abc.com");
}**catch**(**Exception** e) {**System.out.println**(e.getMessage());}
}}

Sortie:

Connecting to Google.com

Accès interdit.



Protection Proxy: la classe Réelle

```
public class RealInter implements Internet{  
    @Override  
public void connectTo(String serverHost) throws  
Exception{  
    System.out.println("Connecting  
to...."+serverHost);    }}
```

Protection Proxy: la classe Proxy

```
import java.util.ArrayList;
import java.util.List;
public class ProxyInter implements Internet{
private RealInter realInter;
private static List<String> sitesForbid;
static {sitesForbid=new ArrayList<String>();
        sitesForbid.add("abc.com");
        sitesForbid.add("gogl.com");
        @Override
public void connectTo(String serverHost) throws Exception {
if (sitesForbid.contains(serverHost.toLowerCase())) {throw new
Exception("Accès Interdit");}
else {realInter=new RealInter();
        realInter.connectTo(serverHost);}
}}
```

Remote proxy

- Nous allons le voir en TD et en TP avec RMI,(voir le cours attaché sur RMI)